MACHINE LEARNING PRACTICALS

Algorithms and Flowcharts — Practicals 1 to 5

Prepared according to the ML practical topics and code discussed in this chat

Practical	Topic
1	PCA / Feature Transformation — Wine Dataset
2	Uber Fare Prediction — Linear, Ridge and Lasso Regression
3	KNN Classification — Social Network Ads
4	K-Means Clustering — Iris Dataset and Elbow Method
5	Random Forest Classification — Car Evaluation Dataset

Each practical includes an aim, numbered algorithm, flowchart, and a short note about dataset/code details.

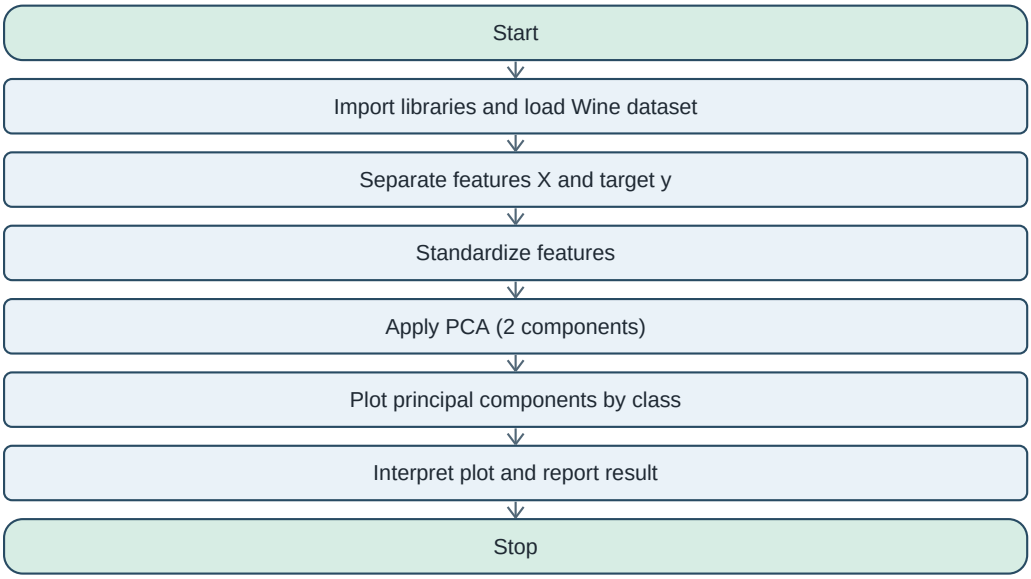
Practical 1: Feature Transformation using PCA on Wine Dataset

Aim: To standardize wine features, reduce dimensionality using Principal Component Analysis (PCA), and visualize how samples separate in the principal-component space.

Algorithm

1. Start.
2. Import the required libraries and load the Wine dataset.
3. Separate input features (X) and target labels (y).
4. Standardize the feature values using StandardScaler.
5. Apply PCA to reduce the standardized features to two principal components.
6. Transform the original features into the two-dimensional PCA space.
7. Plot the transformed samples and use colours to indicate target classes.
8. Interpret the plot and report the observed class separation.
9. Stop.

Flowchart



Note: Check the exact Wine.csv target column: the commonly used UCI Wine Recognition dataset contains three cultivar classes, not red/white wine types.

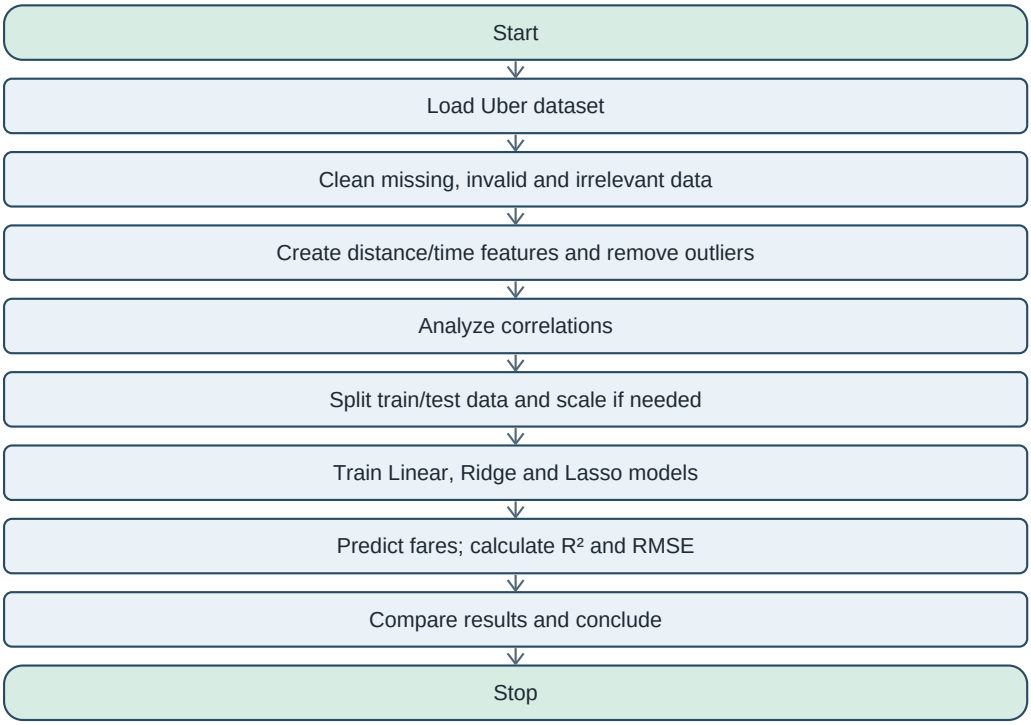
Practical 2: Uber Fare Prediction using Regression

Aim: To preprocess Uber trip data, analyze correlations, and compare Linear Regression, Ridge Regression, and Lasso Regression using R^2 and RMSE.

Algorithm

1. Start.
2. Import libraries and load the Uber fare dataset.
3. Remove irrelevant columns and handle missing or invalid records.
4. Extract useful date/time features and calculate trip distance from coordinates if required.
5. Detect and remove invalid coordinates, unrealistic fares, and distance outliers.
6. Analyze correlations among numeric variables.
7. Split the cleaned dataset into training and testing sets; scale features where appropriate.
8. Train Linear Regression, Ridge Regression, and Lasso Regression models.
9. Predict fares for the test data.
10. Calculate R^2 and RMSE for each model and compare the results.
11. Stop.

Flowchart



Note: Scale features for Ridge/Lasso where appropriate and ensure outlier-removal code drops the intended row indices.

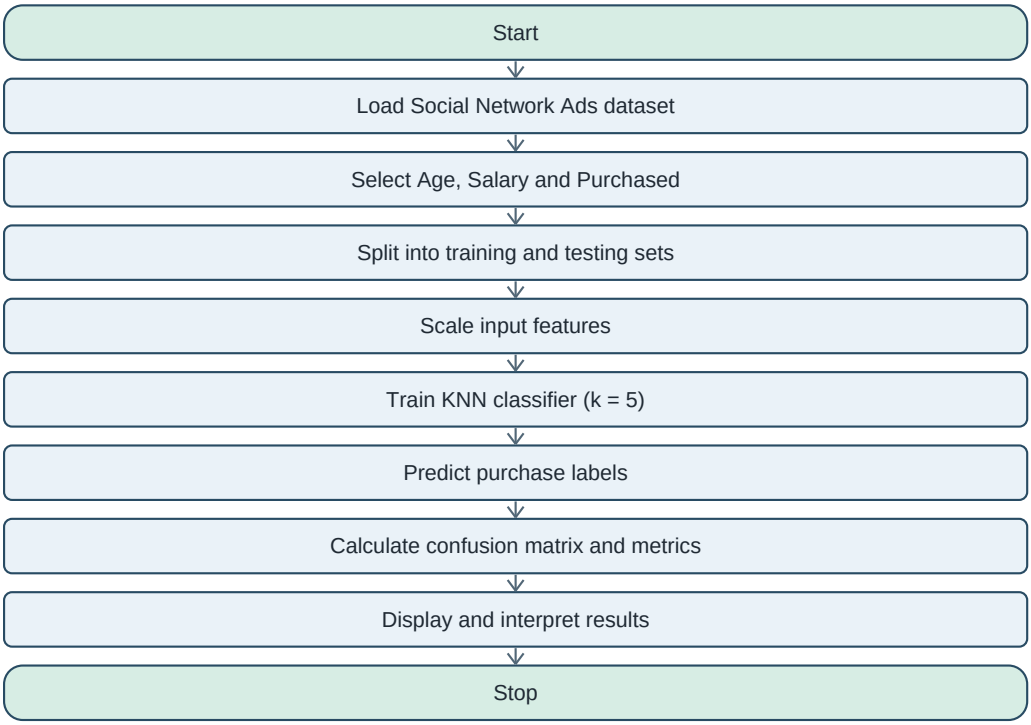
Practical 3: K-Nearest Neighbors (KNN) on Social Network Ads

Aim: To predict whether a user purchases a product using KNN and evaluate the classifier with a confusion matrix, accuracy, error rate, precision, and recall.

Algorithm

1. Start.
2. Import libraries and load Social_Network_Ads.csv.
3. Select Age and EstimatedSalary as input features (X) and Purchased as target (y).
4. Split the data into training and testing sets.
5. Standardize input features using StandardScaler.
6. Initialize a KNN classifier (for example, k = 5).
7. Train the classifier on the training data.
8. Predict purchase labels for the test data.
9. Calculate the confusion matrix, accuracy, error rate, precision, and recall.
10. Display the confusion matrix as a heatmap and interpret the metrics.
11. Stop.

Flowchart



Note: Feature scaling is important for KNN because Age and EstimatedSalary have very different ranges. Fit the scaler only on training data.

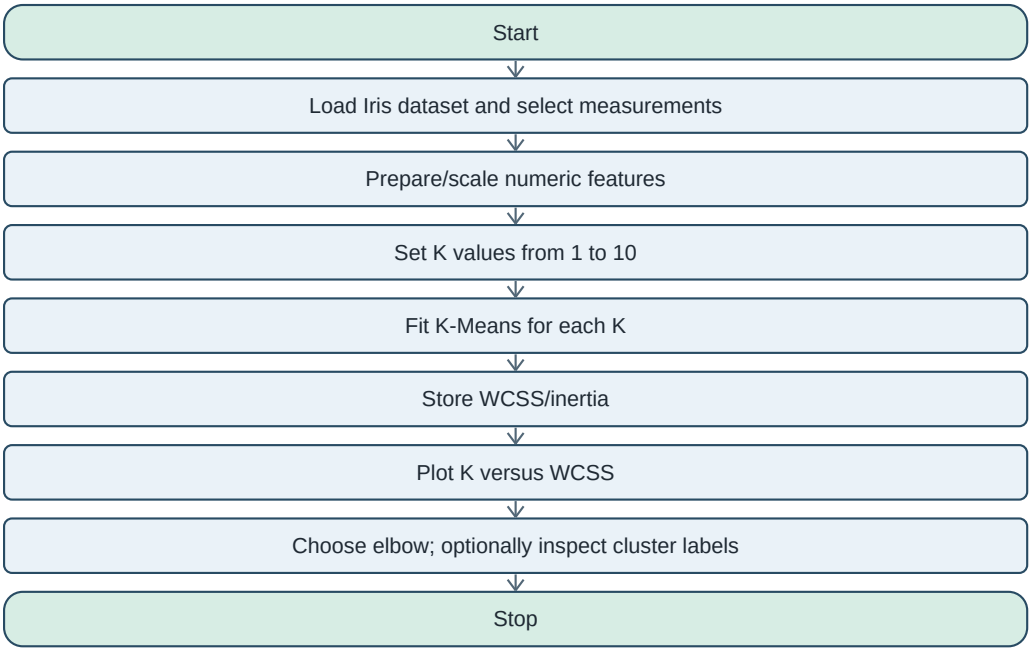
Practical 4: K-Means Clustering and Elbow Method on Iris Dataset

Aim: To apply K-Means clustering to Iris measurements and use the Elbow Method to estimate a suitable number of clusters.

Algorithm

1. Start.
2. Import libraries and load Iris.csv or the scikit-learn Iris dataset.
3. Select the numeric flower measurements and standardize them if appropriate.
4. Create an empty list to store WCSS (model inertia).
5. For each K from 1 to 10, fit a K-Means model.
6. Store the WCSS/inertia value for each K.
7. Plot K against WCSS.
8. Inspect the plot and select the elbow point as the estimated cluster count.
9. Optionally fit K-Means again using the selected K and inspect cluster labels.
10. Stop.

Flowchart



Note: Your code uses scikit-learn's built-in Iris dataset. Use Iris.csv instead only if your instructor specifically requires that file.

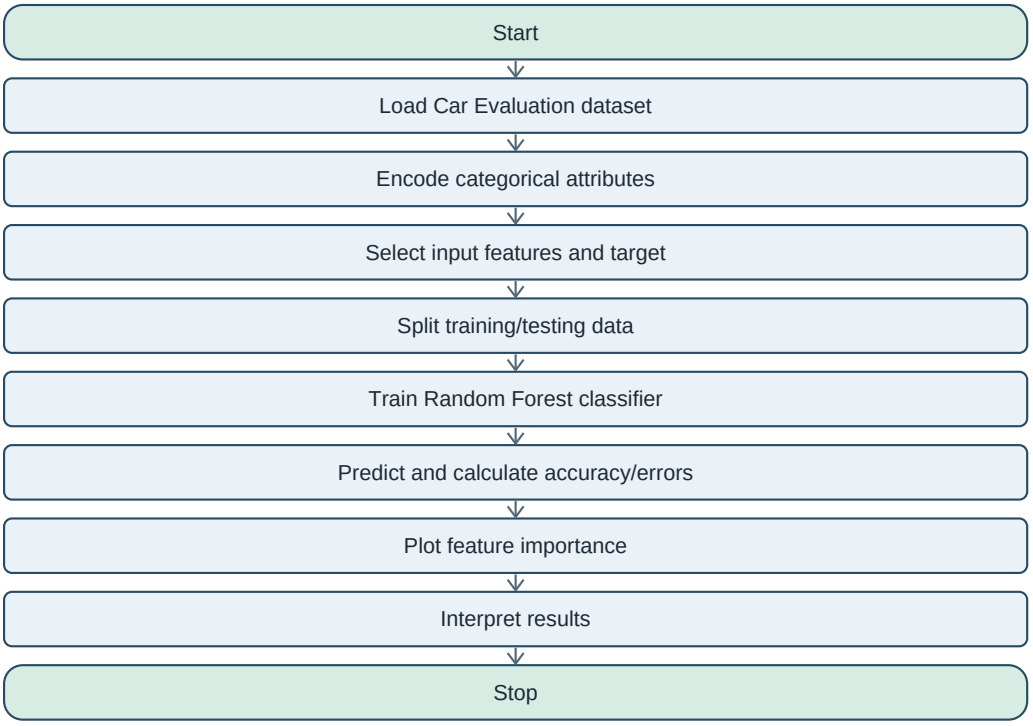
Practical 5: Random Forest Classification on Car Evaluation Dataset

Aim: To train and evaluate a Random Forest classifier on car-evaluation attributes and inspect the importance of input features.

Algorithm

1. Start.
2. Import libraries and load car_evaluation.csv.
3. Encode the categorical attributes into numeric values.
4. Separate input features (X) from the target variable (y).
5. Split the dataset into training and testing sets.
6. Initialize and train a Random Forest Classifier.
7. Predict target labels for the test set.
8. Calculate accuracy and count misclassified samples.
9. Extract and plot feature-importance scores.
10. Interpret the most important feature and stop.

Flowchart



Note: In the standard dataset, 'class' is overall car acceptability and 'safety' is an input feature. To predict safety specifically, use safety as y and exclude it from X.